Practical No. 10

Aim: Write a java program demonstrating collection framework.

Code:

Program 1:

```
import java.util.*;

public class CollectionsMasterDemo {

    public static void main(String[] args) {
        System.out.println("========================================");
        System.out.println("   JAVA COLLECTIONS FRAMEWORK MASTER DEMO");
        System.out.println("========================================\n");

        demoArrayList();
        demoLinkedList();
        demoHashSet();
        demoTreeSet();
        demoPriorityQueue();
        demoArrayDeque();
        demoHashMap();
        demoTreeMap();
    }

    // ------------------------------------------------------------
    // 1. ARRAYLIST (Ordered, Allows Duplicates, Fast Index Access)
    // ------------------------------------------------------------
    private static void demoArrayList() {
        System.out.println("--- 1. ArrayList Operations ---");
        List<String> list = new ArrayList<>();

        // Add elements
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        list.add("Banana"); // Duplicates allowed

        // Insert at specific index
        list.add(1, "Blueberry");

        // Access and Modify
        System.out.println("Element at index 2: " + list.get(2));
        list.set(2, "Blackberry"); // Replace "Banana" with "Blackberry"

        // Search
        System.out.println("Contains 'Apple'? " + list.contains("Apple"));
        System.out.println("Index of 'Cherry': " + list.indexOf("Cherry"));

        // Remove
        list.remove("Blackberry"); // Remove by object
        list.remove(0);            // Remove by index
```

```
    // Iteration
    System.out.print("Remaining ArrayList elements: ");
    for (String item : list) {
        System.out.print(item + " ");
    }
    System.out.println("\n");
}

// ---------------------------------------------------------------
// 2. LINKEDLIST (Doubly Linked List, Efficient Insert/Delete)
// ---------------------------------------------------------------
private static void demoLinkedList() {
    System.out.println("--- 2. LinkedList Operations ---");
    LinkedList<Integer> linkedList = new LinkedList<>();

    // Add elements
    linkedList.add(10);
    linkedList.add(20);
    linkedList.add(30);

    // Deque/First-Last specific methods
    linkedList.addFirst(5);  // Add to head
    linkedList.addLast(40);  // Add to tail

    // Peek (Retrieve without removing)
    System.out.println("First Element (getFirst): " + linkedList.getFirst());
    System.out.println("Last Element (getLast): " + linkedList.getLast());

    // Remove operations
    linkedList.removeFirst(); // Removes 5
    linkedList.removeLast();  // Removes 40

    System.out.println("LinkedList contents: " + linkedList + "\n");
}

// ---------------------------------------------------------------
// 3. HASHSET (Unordered, Unique Elements, O(1) Search)
// ---------------------------------------------------------------
private static void demoHashSet() {
    System.out.println("--- 3. HashSet Operations ---");
    Set<String> set = new HashSet<>();

    // Add elements (Duplicates ignored)
    set.add("Red");
    set.add("Green");
    set.add("Blue");
    boolean isAdded = set.add("Red"); // Returns false (duplicate)

    System.out.println("Was second 'Red' added? " + isAdded);
    System.out.println("Set size: " + set.size());

    // Search & Remove
    if (set.contains("Green")) {
        set.remove("Green");
    }
```

```
    System.out.println("HashSet contents (Unordered): " + set + "\n");
}

// ---------------------------------------------------------------
// 4. TREESET (Sorted, Unique Elements, Red-Black Tree)
// ---------------------------------------------------------------
private static void demoTreeSet() {
    System.out.println("--- 4. TreeSet Operations ---");
    TreeSet<Integer> treeSet = new TreeSet<>();

    treeSet.add(40);
    treeSet.add(10);
    treeSet.add(30);
    treeSet.add(20);

    // Natural sorted order automatically maintained
    System.out.println("Sorted TreeSet: " + treeSet);

    // NavigableSet methods
    System.out.println("First (Lowest): " + treeSet.first());
    System.out.println("Last (Highest): " + treeSet.last());
    System.out.println("HeadSet (< 30): " + treeSet.headSet(30));
    System.out.println("TailSet (>= 20): " + treeSet.tailSet(20));
    System.out.println();
}

// ---------------------------------------------------------------
// 5. PRIORITYQUEUE (Min-Heap / Custom Order)
// ---------------------------------------------------------------
private static void demoPriorityQueue() {
    System.out.println("--- 5. PriorityQueue Operations ---");
    // Default Min-Heap (smallest element has highest priority)
    Queue<Integer> pq = new PriorityQueue<>();

    pq.offer(50);
    pq.offer(10);
    pq.offer(30);

    // Peek top element without removal
    System.out.println("Top element (peek): " + pq.peek());

    // Poll elements in order of priority
    System.out.print("Polling elements from PriorityQueue: ");
    while (!pq.isEmpty()) {
        System.out.print(pq.poll() + " "); // 10, 30, 50
    }
    System.out.println("\n");
}

// ---------------------------------------------------------------
// 6. ARRAYDEQUE (Double-Ended Queue / Stack replacement)
// ---------------------------------------------------------------
private static void demoArrayDeque() {
    System.out.println("--- 6. ArrayDeque (Used as Stack & Queue) ---");
    ArrayDeque<String> stack = new ArrayDeque<>();
```

```
    // LIFO (Stack Behavior)
    stack.push("Bottom");
    stack.push("Middle");
    stack.push("Top");

    System.out.println("Popped from Stack: " + stack.pop()); // Top
    System.out.println("Current Stack Top (peek): " + stack.peek());
    System.out.println();
}

// ---------------------------------------------------------------
// 7. HASHMAP (Key-Value Pairs, Unordered, Fast Lookup)
// ---------------------------------------------------------------
private static void demoHashMap() {
    System.out.println("--- 7. HashMap Operations ---");
    Map<String, Integer> map = new HashMap<>();

    // Put Key-Value pairs
    map.put("Alice", 85);
    map.put("Bob", 92);
    map.put("Charlie", 78);

    // Update value for existing key
    map.put("Alice", 90);

    // Access value by key
    System.out.println("Alice's score: " + map.get("Alice"));
    System.out.println("Default score if absent: " + map.getOrDefault("David", 0));

    // Key/Value checks
    System.out.println("Contains key 'Bob'? " + map.containsKey("Bob"));

    // Iterate through entries
    System.out.println("Iterating over HashMap Entries:");
    for (Map.Entry<String, Integer> entry : map.entrySet()) {
        System.out.println("  " + entry.getKey() + " -> " + entry.getValue());
    }
    System.out.println();
}

// ---------------------------------------------------------------
// 8. TREEMAP (Sorted Key-Value Pairs, Red-Black Tree)
// ---------------------------------------------------------------
private static void demoTreeMap() {
    System.out.println("--- 8. TreeMap Operations ---");
    TreeMap<Integer, String> treeMap = new TreeMap<>();

    treeMap.put(103, "Database");
    treeMap.put(101, "Data Structures");
    treeMap.put(102, "Algorithms");

    // Keys are kept in natural ascending order
    System.out.println("Sorted TreeMap: " + treeMap);

    // NavigableMap operations
```

```
        System.out.println("Lowest Key: " + treeMap.firstKey());
        System.out.println("Highest Key: " + treeMap.lastKey());
        System.out.println("SubMap (101 to 103): " + treeMap.subMap(101, 103));
        System.out.println();
    }
}
```

Output:

```
==========================================
   JAVA COLLECTIONS FRAMEWORK MASTER DEMO
==========================================

--- 1. ArrayList Operations ---
Element at index 2: Banana
Contains 'Apple'? true
Index of 'Cherry': 3
Remaining ArrayList elements: Blueberry Cherry Banana

--- 2. LinkedList Operations ---
First Element (getFirst): 5
Last Element (getLast): 40
LinkedList contents: [10, 20, 30]

--- 3. HashSet Operations ---
Was second 'Red' added? false
Set size: 3
HashSet contents (Unordered): [Red, Blue]
```

```
--- 4. TreeSet Operations ---
Sorted TreeSet: [10, 20, 30, 40]
First (Lowest): 10
Last (Highest): 40
HeadSet (< 30): [10, 20]
TailSet (>= 20): [20, 30, 40]

--- 5. PriorityQueue Operations ---
Top element (peek): 10
Polling elements from PriorityQueue: 10 30 50

--- 6. ArrayDeque (Used as Stack & Queue) ---
Popped from Stack: Top
Current Stack Top (peek): Middle

--- 7. HashMap Operations ---
Alice's score: 90
Default score if absent: 0
Contains key 'Bob'? true
Iterating over HashMap Entries:
  Bob -> 92
```